JAVASCRIPT

LES PROMESSES

No notes on this slide.

JAVASCRIPT

Les promesses

Considérons l'exemple suivant, qui décrit à grands traits le chargement d'une carte

```
load_map_tiles();     // Chargement de la carte, prenant quelques secondes
process_map_tiles();  // Calculs complexes, prenant aussi quelques secondes

/* ... suite du code */
```

Le JavaScript s'exécutant ligne après ligne, tant qu'une fonction n'a pas fini son exécution, aucun autre code JavaScript ne s'exécutera.
Tant que du JavaScript s'exécute, la page est "gelée", et ne réagit pas. Cela est très désagréable pour la personne utilisant la page web.

On aimerait une solution où les opérations longues ne figent pas la page, c'est à dire où ces opérations s'exécutent *en parallèle* du reste du programme. On appelle également cela une *exécution asynchrone*.

No notes on this slide.

JAVASCRIPT

Les promesses

Les promesses nous permettent de faire cela

```
new Promise(() => {  // Création d'une promesse, l'exécution principale continue instantaném
    load_map_tiles();
    process_map_tiles();
    console.log("Tuiles chargées");
});
// La carte est prête, on peut interagir avec l'application
console.log("La carte est interactive");
```

Une `Promise` est un objet prenant en paramètre une fonction. Cette fonction s'exécutera *en parallèle* du reste du code.

La création d'une promesse ne prend presque aucun temps, dans l'exemple ci-dessus on pourra interagir avec la carte immédiatement.

Le contenu de la promesse s'exécutera parallèlement au reste du code.

C'est la machine elle-même qui déterminera l'ordre d'exécution entre le contenu d'une promesse et le reste du code, Cela dépend de beaucoup de paramètres et est déterminé par le processeur de l'ordinateur lui-même. La personne qui écrit le code ne contrôle pas cela. On considère donc que cela est aléatoire, plusieurs exécutions d'un même programme, même dans des conditions similaires, peuvent avoir des ordres d'exécution différents.

JAVASCRIPT

Les promesses

Attention, les promesses ne sont pas magiques

```
let result;
new Promise(() => {
    result = complex_computations();
    console.log("Log2 : ", result);
});
console.log("Log1 : ", result);
// Reste du code
```

Ce code peut afficher d'abord le Log2 puis le Log1, ou le Log1 puis le Log2, cela est indéfini.

Dans ce deuxième cas, `result` peut ne pas être encore assigné et `Log1 : undefined` sera affiché. Si `complex_computations` prend du temps à s'exécuter, c'est ce qui est presque certain de se produire.

Conclusion : **Quand une valeur est assignée dans une promesse, ne pas l'utiliser en dehors.** Sinon cela peut provoquer des comportement inattendus.

Les problèmes qui surviennent à cause de plusieurs fils d'exécution qui créent des comportements inattendus sont appelés des problèmes de concurrence.
Ils sont parmi les plus difficiles à corriger, car leur apparition dépend de l'ordre dans lequel les instructions s'exécutent, ce qui varie à chaque exécution. Le mieux est donc de faire attention à ne pas créer ce genre de bugs...

JAVASCRIPT

Les promesses

De même, ceci est faux

```
let result;
new Promise(() => {
    result = complex_computations();
});
while(result == undefined){} // Ne jamais attendre avec une boucle
console.log(result);
// Reste du code
```

Ce code est valide, mais rend la promesse inutile, car on attend qu'elle soit résolue en bloquant l'exécution du programme.

Il ne faut jamais bloquer l'exécution du JavaScript de la sorte.

No notes on this slide.

JAVASCRIPT

Les promesses

Pour résoudre ce problème on peut soit placer le `console.log` dans la promesse, soit utiliser sa fonction `then`

```
let result;
new Promise(() => {
    result = complex_computations();
    console.log(result);
});
// Reste du code
```

```
let result;
const promesse = new Promise(() => {result = complex_computations();} );
promesse.then(() => console.log(result) )
// Reste du code
```

La fonction `then` prend en paramètre une fonction à exécuter lorsque la promesse sera *résolue*, c'est à dire exécutée jusqu'au bout.
Évidemment, l'affichage ne se fera pas immédiatement, mais seulement quand `complex_computations()` aura fini sont exécution.

No notes on this slide.

JAVASCRIPT

Les promesses

Il est très courant de définir des fonctions retournant des promesses. C'est le cas de toutes les opérations qui peuvent prendre du temps : chargement de fichier distant, calcul complexe, traitement de grande quantité de données, etc.

C'est aussi le cas de fonctions de l'API de base en JavaScript, comme `fetch`

```
const url = "https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3d
// Récupère le fichier à l'URL indiquée, dans une promesse
const requestPromise = fetch(url)

// Quand la requête est résolue, elle retourne une seconde promesse
// Cette seconde promesse sert à transformer le texte en objet JSON
const dataPromise = requestPromise.then(request => request.json())

// Finalement, quand cette seconde promesse est résolue, on affiche le résultat
dataPromise.then(data => console.log(data))

// Le code se trouvant ici ne sera pas bloqué pas les requêtes au dessus
```

No notes on this slide.

JAVASCRIPT

Les promesses

On écrira typiquement ce type de fonction

```
// Fonction isolant le titre d'un jeu de donnée chargé depuis une adresse externe
function get_data_title(url){
    return new Promise( process_title => { // Cette fonction ne fait que retourner une prome
        return fetch(url)
            .then(request => request.json())
            .then(data => process_title(data['title']))
    })
}

const url = "https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3c
get_data_title(url).then(title => console.log("Titre : " + title))
// Affichera "Titre : swissSURFACE3D Raster, the digital surface model of Switzerland"

// Et le reste du code ne sera pas bloqué par toutes ces opérations
```

Par contre, ce type de code devient difficile à comprendre. Il existe une syntaxe plus simple

No notes on this slide.

JAVASCRIPT

async / await

L'exemple précédent peut être réécris de manière équivalente

```
// Fonction isolant le titre d'un jeu de donnée chargé depuis une adresse externe
async function get_data_title(url){
    const request = await fetch(url);
    const data = await request.json();
    return data['title'];
}

const url = "https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3c
get_data_title(url).then(title => console.log("Titre : " + title))
// Affichera "Titre : swissSURFACE3D Raster, the digital surface model of Switzerland"

// Et le reste du code ne sera pas bloqué par toutes ces opérations
```

Le mot clé `async` se place devant une fonction et lui fait retourner une promesse.

Le mot clé `await` indique que le reste du code s'exécutera après la résolution de la promesse qu'il précède

No notes on this slide.

JAVASCRIPT

async / await

Attention aux points suivants avec cette syntaxe :

- Une fonction `async` (dite *fonction asynchrone*) n'est en réalité qu'une fonction retournant une promesse.
- Le mot clé `async` peut s'utiliser devant toute fonction (anonyme ou fléchée également)
- Le mot clé `await` n'est qu'une manière simplifiée de placer le reste de la fonction dans le `.then` de la promesse qu'il précède
- `await` ne peut être utilisé que dans des fonctions asynchrones. Sans quoi cela bloquerait toute la page

No notes on this slide.

JAVASCRIPT

Promesses - Références

Site officiel : https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Promise

W3Schools: https://www.w3schools.com/Js/js_promise.asp

No notes on this slide.